

Tem 4: Backtracking

1) Te dă o listă formată din nr. întregi distincte. Te se genereze toate submultimile ordonate cu aspect de vale. ($\searrow \nearrow$)

Ex: $[5, 4, 3, 6, 7, -1, -2, 3] \Rightarrow [5, 4, 6], [6, 4, 5]$

$[-1, -3, 3, 4, 5, 7]$

Sol: - o sol. posibilă este generarea tuturor submultimilor ordonate, urmând de filtrarea celor cu aspect de vale (poate, mai mic la stamen, nu e eficient) (CAOACIU)

- o soluție mai eficientă presupune combinarea generării soluției cu verificarea validității

- avem o soluție candidat C (listă) în care obținem elemente cât timp soluția este validă

- vom adăuga elemente la începutul listei C

$0 \searrow \nearrow 1$

- variabilă auxiliară $flag < 0$ - parte stângă

- variabilă auxiliară $flag$ $\begin{cases} 0 - \text{poate stănga} \\ 1 - \text{poate dreapta} \end{cases}$

$C_1 \dots C_m$

- generăm din element E din lista originală

- $flag = 1, E < C_1$ 
 - $\rightarrow$ adăugăm E la începutul lui C și continuăm generarea cu $flag = 1$
- $flag = 0, E < C_1$ 
 - $\rightarrow$ nu adăugăm E
- $flag = 1, E > C_1$ 
 - $\rightarrow$ adăugăm E și continuăm cu $flag = 0$
- $flag = 0, E > C_1$ 
 - $\rightarrow$ adăugăm E , $flag = 0$

- C este o ad. finală când $flag = 0$

- la initializarea lui C , vom genera 2 elem E_1, E_2 , a.s. $E_1 < E_2$

prescrie re determinat care generarea, pe rând, toate elementele dintr-o listă

$[1, 2, 3] \rightarrow 1,$
 $2,$

(re determinat $\xrightarrow{3}$ nu mai putem același)
candidat $(l_1, \dots, l_n) = 1. l_1$ doar $n \geq 1$
model matematic

$\text{candidat}(l_1, \dots, l_n) =$ 1. l_1 dacă $n \geq 1$
model matematic

2. $\text{candidat}(l_2, \dots, l_n)$, dacă $n \geq 1$

$\text{candidat}([H|_], H).$

$\text{candidat}([_ | T], E) :-$

$\text{candidat}(T, E).$

model de flux : $(i, 0), (i, i)$

$(i, 0)$

$\text{candidat}([1, 2, 3], E).$

$E = 1$ (opțiune j)

$\rightarrow j \Rightarrow \text{candidat}([2, 3], E)$

$E = 2$

$\rightarrow j \Rightarrow \text{candidat}([3], E)$

$E = 3$

$\rightarrow j \Rightarrow \text{candidat}([], E)$

false

$\text{candidat}([1, 2, 1, 1], 1)$

true

true

true

false

generare_vale ($l_1, \dots, l_n, \text{flag}, C_1, \dots, C_m$) =

1. $C_1 \dots C_n, \text{flag} = 0$

2. $\text{generare_vale}(l_1, \dots, l_n, 0, e \oplus C_1, C_2, \dots, C_m)$, unde $e = \text{candidat}(l_1, \dots, l_n)$
 dacă $e > C_1, e \notin C_1, \dots, C_n$

3. $\text{generare_vale}(l_1, \dots, l_n, 1, e \oplus C_1, \dots, C_m)$, unde $e = \text{candidat}(l_1, \dots, l_n)$

% model de flux: $(i, j, i, 0), (i, j, i, j)$

dacă $\text{flag} = 1, e < C_1$

$\text{generare_vale}(-, 0, C, C)$.

$\text{generare_vale}(L, -, [C_1|C],$

$\text{candidat}(L, E), \text{ } \text{ } (i, 0)$

$E > C_1,$

!

$\text{ } (i, j) \text{ nat}(\text{candidat}([C_1|C], E)),$

$\text{generare_vale}(L, 0, [E, C_1|C], R).$

$\text{generare_vale}(L, 1, [C_1|C], R); -$

$\text{candidat}(L, E),$

$E < C_1,$

$\text{generare_vale}(L, 1, [E, C_1|C], R).$

$\text{generare_main}(l_1, \dots, l_n) =$

$\text{generare_vale}(l_1, \dots, l_n, 1, [e_1, e_2])$

unde $e_1 = \text{candidat}(l_1, \dots, l_n),$

$- e_2 = \text{candidat}(l_1, \dots, l_n),$

$e_1 < e_2$

% model de flux: $(i, 0)$ nedeterminist, (i, i)

(verificăm ca rezultatul este o
 vale core se obține din L,
 determinist)

$\text{generare_main}(L, R); -$

$\text{candidat}(L, E_1),$

$\text{candidat}(L, E_2).$

candidat(L, E_1),

candidat(L, E_2),

$E_1 < E_2$,

genereare_vale($L, 1, [E_1, E_2], R$).

determinist)

predicat care selectează toate soluțiile

(COLOCUVU !!!) într-o listă

$\text{main}(l_1, \dots, l_n) = \bigcup_{(n_{\text{minim}})} \text{genereare_main}(l_1, l_n)$

cod: $\rightarrow$ % model de flux: $(i, o), (i, i)$

$\text{main}(L, R) :- \text{findall}(\text{Rez}, \text{genereare_main}(L, \text{Rez}), R).$

ce se
selectează?

— varianta alternativă: candidat returnează
un element și listă fără acel element

$[1, 2, 3]$

$E=1$
 $R=[2, 3];$

$E=2$
 $R=[1, 3];$

$E=3$
 $R=[1, 2];$

H

restul listei

$\text{candidat}(l_1, \dots, l_n) = 1. [\underline{l_1}, l_2, \dots, l_n], \text{ dacă } n \geq 1$

$\text{candidat}(l_1, \dots, l_n) = 1. [l_1, l_2, \dots, l_n], \text{ dacă } n \geq 1$
 $2. [e, l_1 \oplus n], \text{ unde } [e, n] = \text{candidat}(l_2, \dots, l_n)$
 $\text{dacă } n \geq 1$

$\text{candidat}([2, 3])$

$\hookrightarrow e=2, n=[3]$

$$1 \oplus [3] = [1, 3]$$

$\text{candidat}([H|T], H, T).$

$\text{candidat}([H|T], E, [H|R]):-$

$\text{candidat}(T, E, R).$

% model de flux: $(\bar{x}_j, \theta, \theta), (\bar{x}_j, \bar{x}_j, \bar{x}_j)$
 $(\bar{x}_j, \theta, \bar{x}_j), (\bar{x}_j, \bar{x}_j, \theta)$

$\rightarrow$ generare - toate treburile modificate a.s.
 apelul recursiv să se efectueze pe lista
 din care am extras elementul

- dacă s-or cere subșiruri cu aspect de
 reale: $[5, 4, 6] \checkmark, [6, 4, 5] \times$

$[5, 4, 3, 6, 7, -1, -2, 3]$ nu sunt ca în ordinea
din listă

$\hookrightarrow$ modificăm candidat să returneze
 elementul și elementele oflate în față lui în
 lista originală